Mastering Microsoft Fabric

A practitioner's guide to building, organizing & operating Fabric well

For data engineers & analysts · the four decisions that matter

Overview session · ~45 min

Why we’re here

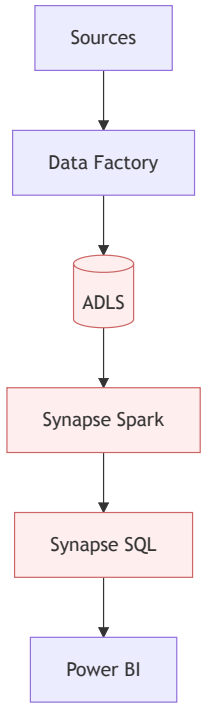
Most Fabric value comes from a handful of **deliberate decisions** — not from knowing every feature.

The old world: a stack of separate products

Before — glue everything yourself

- Azure Data Factory for movement
- Synapse Spark for engineering
- Synapse SQL for warehousing
- Stream Analytics / Data Explorer
- Power BI for BI
- …each with its own storage, security, billing

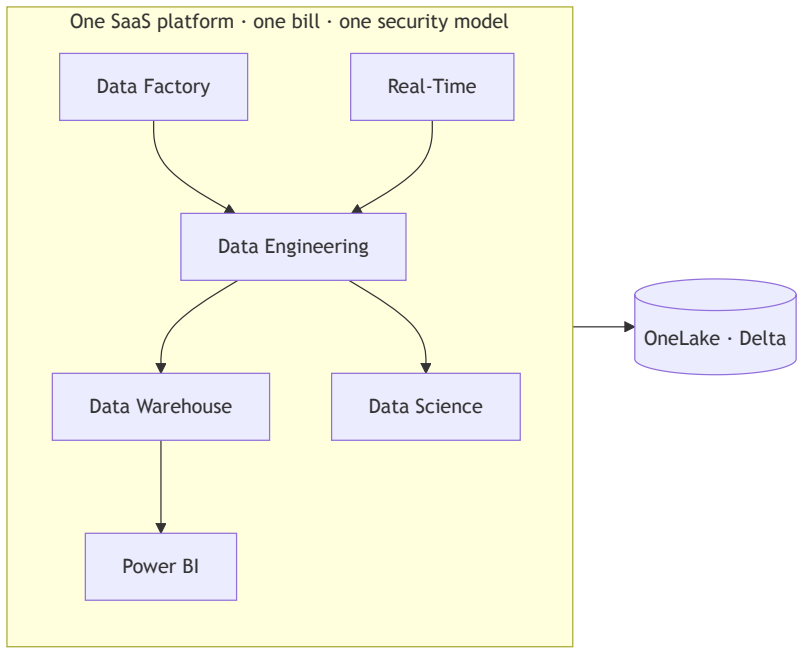
The cost: integration tax, copies everywhere, fragmented governance.



Each hop = a copy, a credential, a failure point.

Microsoft Fabric: one unified, SaaS platform

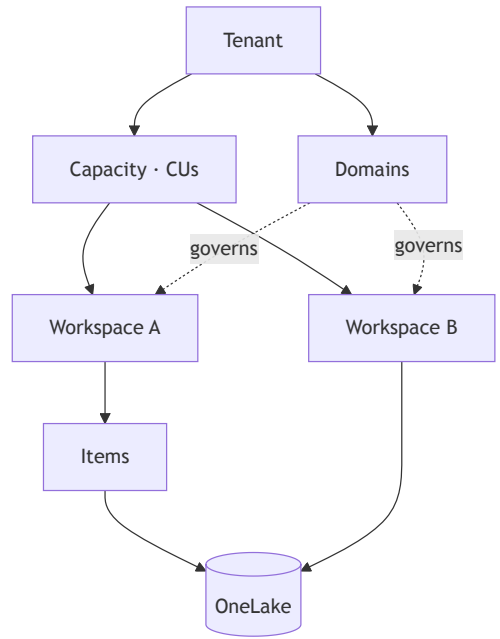
- **SaaS, not PaaS** — buy *capacity*, not clusters & storage accounts
- **OneLake** — one tenant-wide lake; *one copy of data* for every engine
- **Open format** — everything is **Delta-Parquet**; no lock-in



The mental model

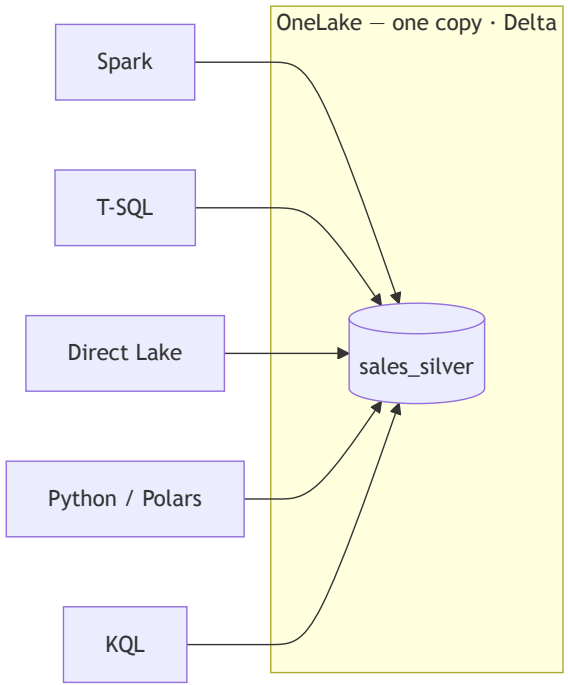
Burn this in — every later choice hangs off it.

- **Tenant** — your org
- **Capacity** — the compute you pay for (CUs)
- **Domain** — governance grouping
- **Workspace** — security · Git · deploy boundary
- **Item** — lakehouse, notebook, model, report…
- **OneLake** — the shared lake under it all



The principle that drives everything

Storage is **shared & open**. Compute is **interchangeable**.



Choosing an engine ≠ choosing where data lives. You rarely copy data again.

The four big decisions

If you make these four deliberately, you can build **anything** in Fabric.

1 · **Lakehouse vs Warehouse**
where data + transform live

2 · **Notebook vs SJD vs Pipeline**
what runs the transform

3 · **Import vs Direct Lake**
how BI reads the model

4 · **Power BI vs Paginated**
how consumers see it

Decision 1 — Lakehouse vs Warehouse

	Lakehouse	**Warehouse**
Persona	Data engineer / scientist	SQL / BI developer
Writes	**Spark** (Delta)	**T-SQL** (DML, procs)
Multi-table transactions	❌	✅
SQL surface	Read-only endpoint	Full read/write
Data shape	Structured + unstructured	Structured

Default → Lakehouse when unsure. Common pattern: **Lakehouse (Spark bronze/silver) + Warehouse (T-SQL gold)** in one workspace.

Decision 2 — Notebook vs Spark Job Definition vs Pipeline

Tool	Choose when
Notebook	Dev, exploration, transforms, ML, orchestrated logic — *the default*
Spark Job Definition	Compiled/JVM batch, fire-and-forget with auto **queue + retry**
Pipeline / Copy job	Orchestrate + bulk move; metadata-driven ingestion
Dataflow Gen2	Low-code Power Query, 150+ connectors
T-SQL (Warehouse)	Set-based SQL, stored procs, COPY INTO / CTAS

⚡ Tip: high-concurrency sessions bill only the initiator — huge for parallel pipeline fan-outs.

Decision 3 — Import vs Direct Lake

- **Import** — full copy in memory; every DAX feature; long refreshes
- **Direct Lake** — reads V-Ordered Delta straight into VertiPaq; **framing** in *seconds*
- **DirectQuery** — live on source; slowest

Default → Direct Lake on well-tuned gold (V-Order + OPTIMIZE).

The nuance that bites people:

- **Direct Lake on OneLake** → *no fallback* (fails hard)
- **Direct Lake on SQL** → *falls back to DirectQuery*

Marco Russo: Direct Lake for big facts, Import for dimensions — as one composite model.

Decision 4 — Power BI report vs Paginated report

	Power BI (interactive)	**Paginated (RDL)**
For	Dashboards, exploration	Print / PDF, operational, pixel-perfect
Large tables	Scroll	**Overflow across pages**
Export	PDF/PPT/Excel (limited)	PDF, **Excel, Word**, CSV…
Tool	Power BI Desktop	Power BI Report Builder

Trigger for paginated: must be printed/PDF'd · grids could overflow · paginated-only features (mail-merge, subreports).

Organize

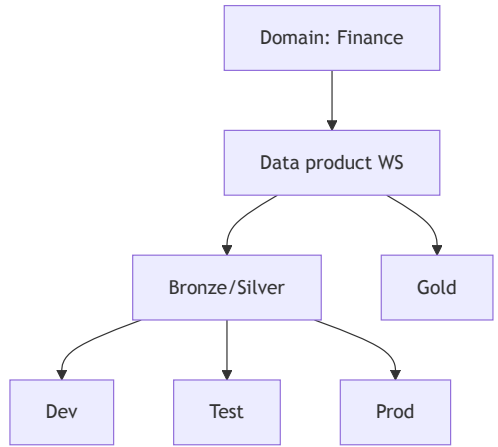
Workspaces, domains & naming — get this right *before* you build.

Workspace topology

Four deployment patterns

1. Monolithic — 1 workspace
2. Multi-workspace, shared capacity
3. **Multi-workspace, separate capacities** ← enterprise default
4. Multiple tenants

Per-environment (Dev/Test/Prod) is mandatory — deployment pipelines require it.



Default: **one workspace per data product, per environment**, domain-aligned.

Domains, roles & naming

Roles (assign to security *groups*) Admin · Member · Contributor · Viewer

⚠️ OneLake data-security applies **only to Viewers** — Admin/Member/Contributor **bypass** it.

⚠️ A **domain is not an access boundary** — it's for governance & discovery.

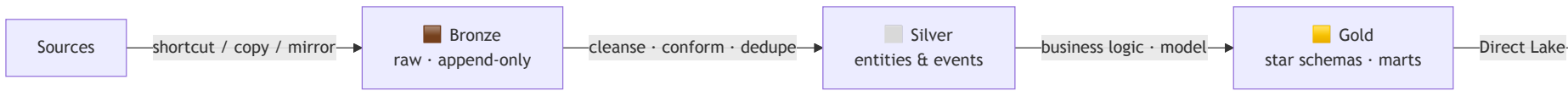
Naming (community standard)

- Workspaces: `FIN-Sales Mart - Gold` *(no env on prod)*
- Items: `LH_STORE_Bronze`, `NB_TRNSF_BronzeToSilver`
- ⚠️ **No env in item names** — deployment pipelines pair *by name*
- Tables: `raw_crm_customers` → `customer` → `dim_customer` / `fact_sales`

Build

Medallion architecture — assembled from lakehouses, schemas & shortcuts.

Bronze · Silver · Gold



Bronze
Never edited — your replay buffer

Silver
Where data becomes trustworthy

Gold
Shaped for consumption

Physical layout: schemas (simple) → separate lakehouses → workspace-per-layer (MS-recommended). Per-environment split is non-negotiable.

The differentiator: Data Products

Stop engineering pipelines. Start declaring products that **survive business change**.

From pipelines to products

"The hard problem isn't moving data — it's surviving business change without losing institutional memory."

Pipeline-first ❌ `Source → Warehouse → dbt → BI` Every link a liability; one perspective baked in.

Product-first ✅ *Declare* inputs · output schema · quality · serving intents. The platform runs it. The engineer is a **domain translator**, not a plumber.

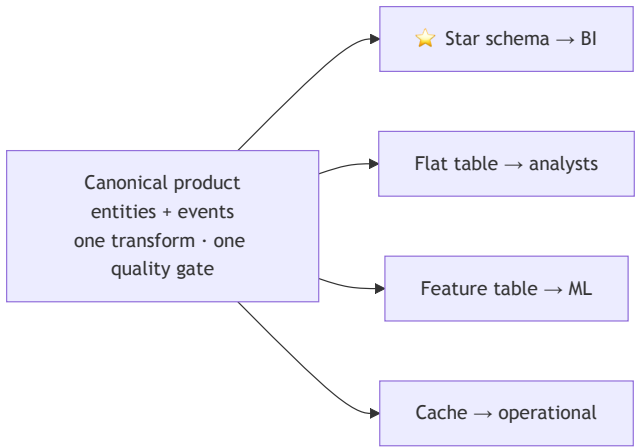
Two primitives + identity separation

- **Entities** — stable identity others reference (`customer_id` survives everything)
- **Events** — immutable, timestamped facts (you can't un-place an order)

The structural invariant:*identity columns are separated from attributes.* Identity is immutable & globally governed; attributes evolve and retire freely.

```
CREATE TABLE silver.customer (
  -- IDENTITY (immutable)
  customer_id      STRING NOT NULL,
  -- ATTRIBUTES (mutable)
  full_name        STRING,
  customer_segment STRING,
  -- AUDIT
  _ingested_at     TIMESTAMP,
  _valid_from      TIMESTAMP
) CLUSTER BY (customer_id);
```

Star schemas are a *projection*, not the architecture



- One transformation, one quality gate → **many disposable projections**
- **SCD type chosen at serving time** — Type 1 or Type 2 from the same data, no new ETL
- Consumers bind to a **contract / semantic intent**, never to your internal columns

Ontology — emergent, governed at the boundaries

- Every product declaring `customer` adds a **node**; every reference adds an **edge** → the ontology *emerges*
- It only self-corrects **within a bounded context** (a domain)
- Across domains, the same word may differ — a **polyseme** needing deliberate governance

In Fabric:

- **Domains** = bounded contexts
- **Purview Unified Catalog** = glossary, lineage, data products
- **Semantic model / Metric sets** = ontology made *queryable* (and, in 2026, fed to AI agents)

Serve

Semantic models, KPIs & reports.

Direct Lake & the Metrics layer

Direct Lake

- V-Order + OPTIMIZE are *mandatory* for good performance
- "Refresh" = **framing** (metadata, seconds)
- Star schema: unique one-side keys, marked date table, no GUIDs/free-text

Three KPI mechanisms

- **Calculation groups** — reuse DAX logic (YoY, YTD)
- **Metric sets** — governed, reusable metric *definitions* (the contract for a KPI)
- **Goals / Scorecards** — track vs targets with status

Prep data **upstream in gold** — not in the model.

Operate

Governance, security, cost & CI/CD.

Security & cost in one slide

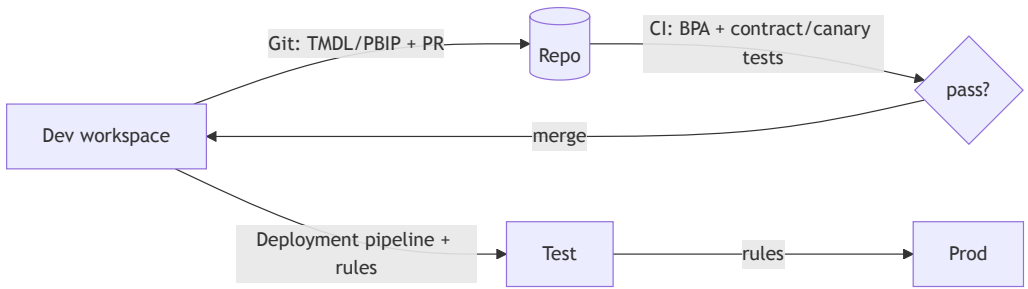
Security layers

- **RLS** rows · **OLS** tables/cols · **CLS** columns
- **OneLake Security** brings CLS to Direct Lake (Viewers only)
- **Sensitivity labels** + **Purview** DLP across the estate
- **Service principals** (SPN) for automation — enable in Tenant settings, add to workspace roles

Capacity & cost

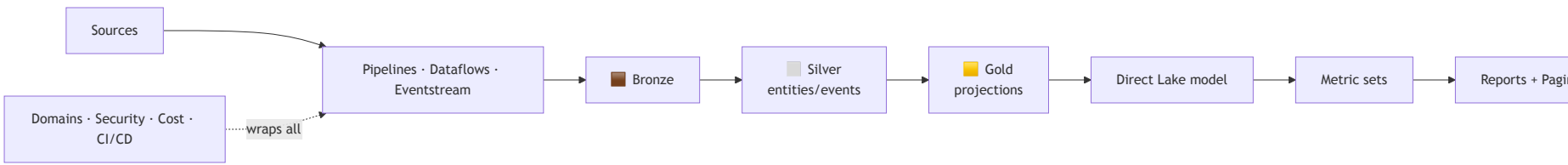
- CUs · **smoothing / bursting / throttling**
- One bad job can throttle *everyone* on a capacity
- **Pause non-prod** off-hours (highest ROI)
- Monitor: **Capacity Metrics app** + **FUAM** (tenant-wide, chargeback)
- Move bursty Spark to **Autoscale Billing**

Ship it — CI/CD



Automate as a **service principal** with `fabric-cicd` / Fabric CLI / Terraform. Keep env **out of item names** so stages auto-pair. Certify prod models.

The reference architecture

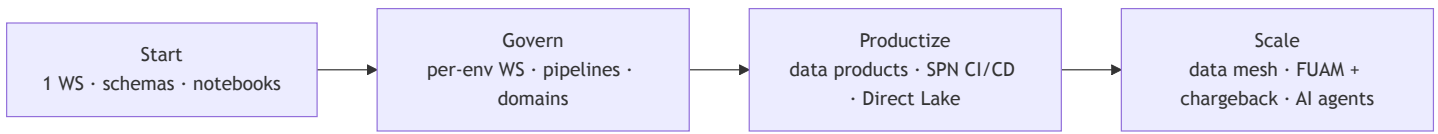


The one rule that prevents the graveyard

Model **entities & events truthfully in silver**,
render **disposable projections in gold**,
and let consumers bind to **contracts, not internals**.

Everything else is mechanics.

Maturity path — don’t boil the ocean



Move up a level when more than one team builds · KPIs diverge · domains publish to each other.

The full course

15 modules · decision tables · hands-on labs · diagrams · the tooling ecosystem

Foundations 00–04	Engineering 05–08	Analytics & BI 09–11
Operate 12–13	Operating model 14	Tooling appendix 99 · fabricstack.dev

📖 Read the full course · 🌐 morse2580.github.io/fabric-course

Thank you

Now go make the four decisions deliberately.

Questions? Start with Module 00 — the mental model.